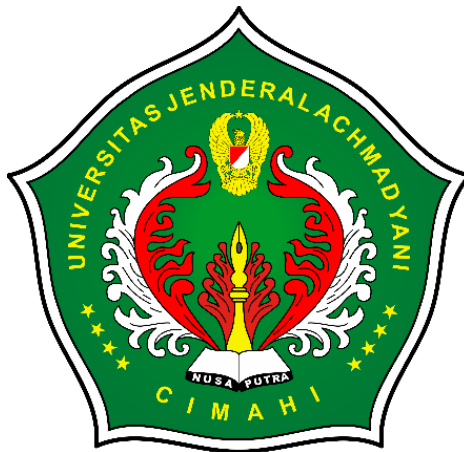


**LAPORAN PRAKTIKUM
STRUKTUR DATA**

**MODUL 6 & 7
QUEUE NON CIRCULAR &
QUEUE CIRCULAR**

**DISUSUN OLEH :
FAISAL FAJARI 2550081098**



**PROGRAM STUDI INFORMATIKA
FAKULTAS SAINS DAN INFORMATIKA
UNIVERSITAS JENDERAL ACHMAD YANI
TAHUN 2026**

DAFTAR ISI

DAFTAR ISI.....	i
DAFTAR GAMBAR.....	ii
BAB I. HASIL PRAKTIKUM.....	1
I.1 Queue Non Circular Alternatif 1.....	1
I.1.A. Source Code.....	1
I.1.B. Screenshot.....	11
I.1.C. Analisis.....	11
I.2 Queue Non Circular Alternatif 2.....	13
I.2.A. Source Code.....	13
I.2.B. Screenshot.....	22
I.2.C. Analisis.....	22
BAB II. TUGAS PRAKTIKUM.....	24
II.1 Queue Circular Array.....	24
II.1.A. Source Code.....	24
II.1.B. Screenshot.....	34
II.1.C. Analisis.....	34
II.2 Queue Circular LinkedList.....	36
II.2.A. Source Code.....	36
II.2.B. Screenshot.....	46
II.2.C. Analisis.....	46
BAB III. KESIMPULAN.....	47

DAFTAR GAMBAR

Gambar 1. 1 AddQueue Non Circular Alternatif_1.....	11
Gambar 1. 2 CetakQueue Non Circular Alternatif_1.....	11
Gambar 1. 3 Dequeue Non Circular Alternatif_1.....	11
Gambar 1. 4 AddQueue Non Circular Alternatif_2.....	22
Gambar 1. 5 CetakQueue Non Circular Alternatif_2.....	22
Gambar 1. 6 Dequeue Non Circular Alternatif_2.....	22
Gambar 2. 1 AddQueue Circular Array.....	33
Gambar 2. 2 CetakQueue Circular Array.....	33
Gambar 2. 3 Dequeue Circular Array.....	33
Gambar 2. 4 AddQueue Circular Linked_List.....	45
Gambar 2. 5 CetakQueue Circular Linked_List.....	45
Gambar 2. 6 Dequeue Circular Linked_List.....	45

BAB I. HASIL PRAKTIKUM

I.1 Queue Non Circular Alternatif 1

I.1.A. Source Code

```
/* Program : boolean.h
Author : 2550081098, Faisal Fajari
Kelas : E
Deskripsi: Header file dari prototype Boolean
Tanggal : 06/04/2026
*/
#ifndef boolean_H
#define boolean_H
#define true 1
#define false 0
#define boolean unsigned char
#endif
```

```

/* Program : queue.h
Author : 2550080198_Faisal Fajari
Kelas : E
Deskripsi: Header file dari prototype queue
Tanggal : 18/5/26
*/

#ifndef _QUEUE_H
#define _ QUEUE _H
#include "boolean.h"
#define Nil 0
#define MaxEl 10
#define idxUndef 999;

#define Head(Q) (Q).HEAD
#define Tail(Q) (Q).TAIL
#define InfoHead(Q) (Q).T[(Q).HEAD]
#define InfoTail(Q) (Q).T[(Q).TAIL]

typedef int infotype;
typedef int address;

typedef struct {
    infotype T[MaxEl+1];
    address HEAD;
    address TAIL;
} Queue;

void CreateQueue(Queue *Q);
int NBElt(Queue Q);
boolean IsQueueEmpty (Queue Q);
boolean IsQueueFull(Queue Q);

void AddQueue (Queue *Q, infotype X);
void DelQueue (Queue *Q, infotype *X);

void PrintQueueInfo (Queue S);

boolean isInfoKetemu(Queue S, infotype X);
address CariElemenQueue(Queue Q, int X);

```

```
#endif
```

```

/* Program : queue.c
Author : 2550080198_Faisal Fajari
Kelas : E
Deskripsi: Body file dari prototype queue
Tanggal : 18/5/26
*/
#include "queue.h"
#include "boolean.h"
#include "stdio.h"

void CreateQueue(Queue *Q){
    Q->HEAD = Nil;
    Q->TAIL = Nil;
    int counter = 0;
    for (counter = 0; counter <= MaxEl; counter++){
        Q->T[counter] = Nil;
    };
};

boolean IsQueueEmpty (Queue Q){
    return (Q.HEAD == Nil);
};

int NBElt(Queue Q){
    return (Q.TAIL);
};

boolean IsQueueFull(Queue Q){
    return (Q.TAIL == MaxEl);
};

void AddQueue (Queue *Q, infotype X){

    if (IsQueueEmpty(*(Q))){
        Q->HEAD = 1;
        Q->TAIL = 1;
        Q->T[Q->TAIL] = X;
        return;
    };
};

```

```

        if (!IsQueueFull(*(Q))){
            Q->TAIL = Q->TAIL + 1;
            Q->T[Q->TAIL] = X;
            return;
        };

        printf("\nQueue Penuh! \n");
    };

void DelQueue (Queue *Q, infotype *X){
    if (IsQueueEmpty(*(Q))){
        return;
    };

    *X = Q->T[Q->HEAD];
    int counter = 0;
    for (counter = Q->HEAD; counter <= Q->TAIL; counter++){
        if (counter == Q->TAIL){
            Q->T[counter] = Nil;
        };
        Q->T[counter] = Q->T[counter + 1];
    };

    Q->TAIL = Q->TAIL - 1;

    if (Q->TAIL == 0){
        Q->HEAD = 0;
    };
    return;
}

void PrintQueueInfo (Queue S){
    if (IsQueueEmpty(S)){
        printf("\nQueue Kosong! \n");
        return;
    };

    printf("Isi Queue : ");

```



```

        int counter = 0;
        for (counter = S.HEAD; counter <= S.TAIL; counter++){
            printf("%d ", S.T[counter]);
        };
        printf("\n");

};

boolean isInfoKetemu(Queue S, infotype X){
    if (IsQueueEmpty(S)){
        return false;
    };

    int counter = 0;
    for (counter = S.HEAD; counter <= S.TAIL; counter++){
        if (S.T[counter] == X){
            return true;
        };
    };

    return false;
};

address CariElemenQueue(Queue Q, int X){
    if(IsQueueEmpty(Q)){
        return idxUndef;
    };

    int counter = 0;
    for (counter = Q.HEAD; counter <= Q.TAIL; counter++){
        if (Q.T[counter] == X){
            return counter;
        };
    };

    return idxUndef;
};

```



```

/* Program : main.c
Author : 2550080198_Faisal Fajari
Kelas : E
Deskripsi: Main Driver file dari prototype queue
Tanggal : 18/5/26
*/

#include <stdio.h>
#include "queue.h"
#include <stdlib.h>

int main(){
    Queue Q;
    CreateQueue(&Q);
    int pilihan = 0;
    char fix;

    do {
        system("clear");
        system("cls");
        printf("1. AddQueue\n");
        printf("2. DelQueue\n");
        printf("3. CetakQueue\n");
        printf("Pilihan : ");
        scanf("%d",&pilihan);

        switch (pilihan) {
            case 1:
                printf("\n=== AddQueue ===\n");
                while (1){
                    printf("Masukan Element : ");
                    scanf("%d", &pilihan);

                    if (IsQueueFull(Q)){
                        printf("Queue Penuh...");
                        getchar();
                        getchar();
                        break;
                    }
                };
            case 2:
                printf("\n=== DelQueue ===\n");
                while (1){
                    printf("Masukan Element : ");
                    scanf("%d", &pilihan);

                    if (IsQueueEmpty(Q)){
                        printf("Queue Kosong...");
                        getchar();
                        getchar();
                        break;
                    }
                };
            case 3:
                printf("\n=== CetakQueue ===\n");
                while (1){
                    printf("Masukan Element : ");
                    scanf("%d", &pilihan);

                    if (IsQueueEmpty(Q)){
                        printf("Queue Kosong...");
                        getchar();
                        getchar();
                        break;
                    }
                };
            default:
                printf("Pilihan Salah\n");
                break;
        }
    } while (fix != 'q');
}

```

```

        AddQueue(&Q, pilihan);

        printf("Lagi (y/n) : ");
        scanf(" %c", &fix);

        if (fix == 'n' || fix == 'N'){
            break;
        };
    };
    break;

case 2:
    printf("\n=== DelQueue ===\n");
    PrintQueueInfo(Q);
    while (1){

        printf("\nDequeue (y/n)? : ");
        scanf(" %c", &fix);

        if (fix == 'y' || fix == 'Y'){
            DelQueue(&Q, &pilihan);
        } else {
            break;
        };

        printf("Element yg dihapus : %d\n",
pilihan);

        PrintQueueInfo(Q);

        if (IsQueueEmpty(Q)){
            printf("Tekan apa saja..");
            getchar();
            getchar();
            break;
        };

    };
};

```

```
        break;

        case 3:
            printf("\n");
            PrintQueueInfo(Q);
            printf("Tekan apa saja...");
            getchar();
            getchar();
            break;
        break;

        default:

            break;

        case 0:

            break;
    }

    } while (pilihan != 0);
};
```

I.1.B. Screenshot

```
C:\Windows\system32\cmd.e  X  +  v
== Queue Non Circular Alternatif 1 ==
1. AddQueue
2. DelQueue
3. CetakQueue
Pilihan : 1

=== AddQueue ===
Masukan Element : 20
Lagi (y/n) : y
Masukan Element : 40
Lagi (y/n) : y
Masukan Element : 60
Lagi (y/n) : |
```

Gambar 1. 1 AddQueue Non Circular Alternatif_1

```
C:\Windows\system32\cmd.e  X  +  v
== Queue Non Circular Alternatif 1 ==
1. AddQueue
2. DelQueue
3. CetakQueue
Pilihan : 3

Isi Queue : 20 40 60
Tekan apa saja...|
```

Gambar 1. 2 CetakQueue Non Circular Alternatif_1

```
C:\Windows\system32\cmd.e  X  +  v
== Queue Non Circular Alternatif 1 ==
1. AddQueue
2. DelQueue
3. CetakQueue
Pilihan : 2

=== DelQueue ===
Isi Queue : 20 40 60

Dequeue (y/n)? : y
Element yg dihapus : 20
Isi Queue : 40 60

Dequeue (y/n)? : y
Element yg dihapus : 40
Isi Queue : 60

Dequeue (y/n)? : |
```

Gambar 1. 3 Dequeue Non Circular Alternatif_1

I.1.C. Analisis

Pada Queue Non Circular Alternatif 1, antrean menggunakan array statis dengan posisi HEAD yang selalu dipaksa berada di indeks awal (indeks 1). Ketika fungsi AddQueue dipanggil dalam kondisi kosong, HEAD dan TAIL diatur ke indeks 1, sedangkan pada kondisi antrean terisi, pointer TAIL akan maju satu langkah sebelum data baru dimasukkan. Sementara itu, fungsi DelQueue bekerja dengan mengambil data di HEAD, lalu memaksa seluruh elemen di belakangnya untuk bergeser satu langkah ke depan menggunakan perulangan for agar posisi depan tidak kosong, sebelum akhirnya mengurangi nilai TAIL sebesar 1. Sistem pergeseran elemen ini membuat pemanfaatan ruang array menjadi optimal di bagian

depan, tetapi mengorbankan kecepatan eksekusi program karena komputer harus memproses ulang posisi setiap sisa elemen data setiap kali terjadi penghapusan.

I.2 Queue Non Circular Alternatif 2

I.2.A. Source Code

```
/* Program : boolean.h
Author : 2550081098, Faisal Fajari
Kelas : E
Deskripsi: Header file dari prototype Boolean
Tanggal : 06/04/2026
*/

#ifndef boolean_H
#define boolean_H
#define true 1
#define false 0
#define boolean unsigned char
#endif
```



```

/* Program : queue.h
Author : 2550080198_Faisal Fajari
Kelas : E
Deskripsi: Header file dari prototype queue
Tanggal : 18/5/26
*/

#ifndef _QUEUE_H
#define _ QUEUE _H
#include "boolean.h"
#define Nil 0
#define MaxEl 10
#define idxUndef 999

#define Head(Q) (Q).HEAD
#define Tail(Q) (Q).TAIL
#define InfoHead(Q) (Q).T[(Q).HEAD]
#define InfoTail(Q) (Q).T[(Q).TAIL]

typedef int infotype;
typedef int address;

typedef struct {
    infotype T[MaxEl+1];
    address HEAD;
    address TAIL;
} Queue;

void CreateQueue(Queue *Q);
int NBElt(Queue Q);

boolean IsQueueEmpty (Queue Q);
boolean IsQueueFull(Queue Q);

void AddQueue (Queue *Q, infotype X);
void DelQueue (Queue *Q, infotype *X);

void PrintQueueInfo (Queue S);

boolean isInfoKetemu(Queue S, infotype X);

```

```
address CariElemenQueue(Queue Q, int X);
```

```
#endif
```

```

/* Program : queue.c
Author : 2550080198_Faisal Fajari
Kelas : E
Deskripsi: Body file dari prototype queue
Tanggal : 18/5/26
*/

#include "queue.h"
#include "boolean.h"
#include "stdio.h"
#define Nil 0
#define MaxEl 10

void CreateQueue(Queue *Q){
    Q->HEAD = Nil;
    Q->TAIL = Nil;
    int counter = 0;
    for (counter = 0; counter <= MaxEl; counter++){
        Q->T[counter] = Nil;
    };
};

boolean IsQueueEmpty (Queue Q){
    return ((Q.HEAD > Q.TAIL) || ((Q.HEAD == Nil) && (Q.TAIL
== Nil))) );
};

int NBElt(Queue Q){
    if (IsQueueEmpty(Q)){
        return 0;
    } else {
        return (Q.TAIL - Q.HEAD + 1);
    }
};

boolean IsQueueFull(Queue Q){
    return (Q.TAIL == MaxEl);
};

void AddQueue (Queue *Q, infotype X){

```

```

        if (IsQueueEmpty(*(Q))){
            Q->HEAD = 1;
            Q->TAIL = 1;
            Q->T[Q->TAIL] = X;
            return;
        };

        if (!IsQueueFull(*(Q))){
            Q->TAIL = Q->TAIL + 1;
            Q->T[Q->TAIL] = X;
            return;
        };

        printf("\nQueue Penuh! \n");
    };

void DelQueue (Queue *Q, infotype *X){
    if (IsQueueEmpty(*(Q))){
        printf("\nQueue Kosong! \n");
        return;
    };

    *X = Q->T[Q->HEAD];
    Q->T[Q->HEAD] = Nil;
    Q->HEAD = Q->HEAD + 1;

    if (Q->HEAD > Q->TAIL){
        CreateQueue(Q);
    };

    return;
}

void PrintQueueInfo (Queue S){
    if (IsQueueEmpty(S)){
        printf("\nQueue Kosong! \n");
        return;
    };
};

```

```

        printf("Isi Queue : ");
        int counter = 0;
        for (counter = S.HEAD; counter <= S.TAIL; counter++){
            printf("%d ", S.T[counter]);
        };
        printf("\n");

};

boolean isInfoKetemu(Queue S, infotype X){
    if (IsQueueEmpty(S)){
        return false;
    };

    int counter = 0;
    for (counter = S.HEAD; counter <= S.TAIL; counter++){
        if (S.T[counter] == X){
            return true;
        };
    };

    return false;
};

address CariElemenQueue(Queue Q, int X){
    if(IsQueueEmpty(Q)){
        return idxUndef;
    };

    int counter = 0;
    for (counter = Q.HEAD; counter <= Q.TAIL; counter++){
        if (Q.T[counter] == X){
            return counter;
        };
    };

    return idxUndef;
};

```

```

/* Program : main.c
Author : 2550080198_Faisal Fajari
Kelas : E
Deskripsi: Main Driver file dari prototype queue
Tanggal : 18/5/26
*/

#include <stdio.h>
#include "queue.h"
#include <stdlib.h>

int main(){
    Queue Q;
    CreateQueue(&Q);
    int pilihan = 0;
    char fix;

    do {
        system("clear");
        system("cls");
        printf("1. AddQueue\n");
        printf("2. DelQueue\n");
        printf("3. CetakQueue\n");
        printf("Pilihan : ");
        scanf("%d",&pilihan);

        switch (pilihan) {
            case 1:
                printf("\n=== AddQueue ===\n");
                while (1){
                    printf("Masukan Element : ");
                    scanf("%d", &pilihan);

                    if (IsQueueFull(Q)){
                        printf("Queue Penuh...");
                        getchar();
                        getchar();
                        break;
                    }
                }
            };
        }
    } while (fix != 'q');
}

```

```

        AddQueue(&Q, pilihan);

        printf("Lagi (y/n) : ");
        scanf(" %c", &fix);

        if (fix == 'n' || fix == 'N'){
            break;
        };
    };
    break;

case 2:
    printf("\n=== DelQueue ===\n");
    PrintQueueInfo(Q);
    while (1){

        printf("\nDequeue (y/n)? : ");
        scanf(" %c", &fix);

        if (fix == 'y' || fix == 'Y'){
            DelQueue(&Q, &pilihan);
        } else {
            break;
        };

        printf("Element yg dihapus : %d\n",
pilihan);

        PrintQueueInfo(Q);

        if (IsEmpty(Q)){
            printf("Tekan apa saja..");
            getchar();
            getchar();
            break;
        };
    };

```

```

        };
        break;

        case 3:
            printf("\n");
            PrintQueueInfo(Q);
            printf("Tekan apa saja...");
            getchar();
            getchar();
            break;
        break;

        default:

            break;

        case 0:

            break;
    }

    } while (pilihan != 0);
};

```


I.2.B. Screenshot



```
C:\Windows\system32\cmd.e. X + v
== Queue Non Circular Alternatif 2 ==
1. AddQueue
2. DelQueue
3. CetakQueue
Pilihan : 1

=== AddQueue ===
Masukan Element : 20
Lagi (y/n) : y
Masukan Element : 90
Lagi (y/n) : y
Masukan Element : 8
Lagi (y/n) : |
```

Gambar 1. 4 AddQueue Non Circular Alternatif_2



```
C:\Windows\system32\cmd.e. X + v
== Queue Non Circular Alternatif 2 ==
1. AddQueue
2. DelQueue
3. CetakQueue
Pilihan : 3

Isi Queue : 20 90 8
Tekan apa saja...|
```

Gambar 1. 5 CetakQueue Non Circular Alternatif_2



```
C:\Windows\system32\cmd.e. X + v
== Queue Non Circular Alternatif 2 ==
1. AddQueue
2. DelQueue
3. CetakQueue
Pilihan : 2

=== DeQueue ===
Isi Queue : 20 90 8

Dequeue (y/n)? : y
Element yg dihapus : 20
Isi Queue : 90 8

Dequeue (y/n)? : y
Element yg dihapus : 90
Isi Queue : 8

Dequeue (y/n)? : |
```

Gambar 1. 6 Dequeue Non Circular Alternatif_2

I.2.C. Analisis

Pada Queue Non Circular Alternatif 2 menghilangkan proses pergeseran elemen yang lambat dengan cara membiarkan pointer depan (HEAD) ikut bergerak maju. Mekanisme penambahan data pada fungsi AddQueue tetap sama dengan alternatif pertama, yaitu memajukan penunjuk TAIL secara linier ke belakang array. Namun pada fungsi DelQueue, penghapusan elemen dilakukan secara instan tanpa menggeser data; nilai pada HEAD diambil, slotnya dikosongkan dengan nilai Nil, dan pointer HEAD langsung dinaikkan satu indeks ke belakang ($HEAD = HEAD + 1$). Jika seluruh data habis didequeue ($HEAD > TAIL$), sistem akan otomatis mereset antrian ke kondisi awal. Pendekatan ini membuat operasi penghapusan data berjalan sangat cepat $O(1)$, tetapi memicu kelemahan berupa false full (penuh

semu), di mana antrean dianggap penuh saat TAIL mencapai batas memori array maksimum walaupun slot di bagian depan sebenarnya kosong.

BAB II. TUGAS PRAKTIKUM

II.1 Queue Circular Array

II.1.A. Source Code

```
/* Program : boolean.h
Author : 2550081098, Faisal Fajari
Kelas : E
Deskripsi: Header file dari prototype Boolean
Tanggal : 06/04/2026
*/

#ifndef boolean_H
#define boolean_H
#define true 1
#define false 0
#define boolean unsigned char
#endif
```

```

/* Program : queuecircular1.h
Author : 2550080198_Faisal Fajari
Kelas : E
Deskripsi: Header file dari prototype queue
Tanggal : 18/5/26
*/

#ifndef _QUEUE_H
#define _ QUEUE _H
#include "boolean.h"
#define Nil -1
#define MaxEl 10
#define idxUndef 999

#define Head(Q) (Q).HEAD
#define Tail(Q) (Q).TAIL
#define InfoHead(Q) (Q).T[(Q).HEAD]
#define InfoTail(Q) (Q).T[(Q).TAIL]

typedef int infotype;
typedef int address;

typedef struct {
    infotype T[MaxEl+1];
    address HEAD;
    address TAIL;
} Queue;

void CreateQueue(Queue *Q);
int NBElt(Queue Q);

boolean IsQueueEmpty (Queue Q);
boolean IsQueueFull(Queue Q);

void AddQueue (Queue *Q, infotype X);
void DelQueue (Queue *Q, infotype *X);

void PrintQueueInfo (Queue S);

boolean isInfoKetemu(Queue S, infotype X);

```

```
address CariElemenQueue(Queue Q, int X);
```

```
#endif
```

```

/* Program : queuecircular1.c
Author : 2550080198_Faisal Fajari
Kelas : E
Deskripsi: Body file dari prototype queue
Tanggal : 18/5/26
*/

#include "queuecircular1.h"
#include "boolean.h"
#include "stdio.h"
#define Nil -1
#define MaxEl 10

void CreateQueue(Queue *Q){
    Q->HEAD = Nil;
    Q->TAIL = Nil;
    int counter = 0;
    for (counter = 0; counter < MaxEl; counter++){
        Q->T[counter] = Nil;
    };
};

boolean IsQueueEmpty (Queue Q){
    return (Q.HEAD == Nil);
};

int NBElt(Queue Q){
    if (IsQueueEmpty(Q)){
        return 0;
    } else {
        return ((Q.TAIL - Q.HEAD + MaxEl) % MaxEl + 1);
    }
};

boolean IsQueueFull(Queue Q){
    int nextTail = Q.TAIL + 1;
    if (nextTail == MaxEl){
        nextTail = 0;
    };
    return (nextTail == Q.HEAD);
};

```

```

};

void AddQueue (Queue *Q, infotype X){

    if (IsQueueEmpty(*(Q))){
        Q->HEAD = 0;
        Q->TAIL = 0;
        Q->T[Q->TAIL] = X;
        return;
    };

    if (!IsQueueFull(*(Q))){
        Q->TAIL = Q->TAIL + 1;
        if (Q->TAIL == MaxEl){
            Q->TAIL = 0;
        };
        Q->T[Q->TAIL] = X;
        return;
    };

    printf("\nQueue Penuh! \n");
};

void DelQueue (Queue *Q, infotype *X){
    if (IsQueueEmpty(*(Q))){
        printf("\nQueue Kosong! \n");
        return;
    };

    *X = Q->T[Q->HEAD];
    Q->T[Q->HEAD] = Nil;

    if (Q->HEAD == Q->TAIL){
        CreateQueue(Q);
    } else {
        Q->HEAD = Q->HEAD + 1;
        if (Q->HEAD == MaxEl){
            Q->HEAD = 0;
        };
    };
};

```

```

};

return;
}

void PrintQueueInfo (Queue S){
    if (IsQueueEmpty(S)){
        printf("\nQueue Kosong! \n");
        return;
    };

    printf("Isi Queue : ");
    int counter = S.HEAD;
    while (1){
        printf("%d ", S.T[counter]);
        if (counter == S.TAIL) {
            break;
        };
        counter = counter + 1;
        if (counter == MaxEl){
            counter = 0;
        }
    };
    printf("\n");

};

boolean isInfoKetemu(Queue S, infotype X){
    if (IsQueueEmpty(S)){
        return false;
    };

    int counter = S.HEAD;
    while (1){
        if (S.T[counter] == X){
            return true;
        };
        if (counter == S.TAIL){
            break;

```



```

        };
        counter = counter + 1;
        if (counter == MaxEl){
            counter = 0;
        };
    };

    return false;
};

address CariElemenQueue(Queue Q, int X){
    if(IsQueueEmpty(Q)){
        return idxUndef;
    };

    int counter = 0;
    for (counter = Q.HEAD; counter <= Q.TAIL; counter++){
        if (Q.T[counter] == X){
            return counter;
        };
    };

    return idxUndef;
};

```

```

/* Program : main.h
Author : 2550080198_Faisal Fajari
Kelas : E
Deskripsi: Main Driver file dari prototype queue
Tanggal : 18/5/26
*/

#include <stdio.h>
#include "queuecircular1.h"
#include <stdlib.h>

int main(){
    Queue Q;
    CreateQueue(&Q);
    int pilihan = 0;
    char fix;

    do {
        system("clear");
        system("cls");
        printf("1. AddQueue\n");
        printf("2. DelQueue\n");
        printf("3. CetakQueue\n");
        printf("Pilihan : ");
        scanf("%d",&pilihan);

        switch (pilihan) {
            case 1:
                printf("\n=== AddQueue ===\n");
                while (1){
                    printf("Masukan Element : ");
                    scanf("%d", &pilihan);

                    if (IsQueueFull(Q)){
                        printf("Queue Penuh...");
                        getchar();
                        getchar();
                        break;
                    }
                };
            }
    } while (fix != 'q');
}

```

```

        AddQueue(&Q, pilihan);

        printf("Lagi (y/n) : ");
        scanf(" %c", &fix);

        if (fix == 'n' || fix == 'N'){
            break;
        };
    };
    break;

case 2:
    printf("\n=== DelQueue ===\n");
    PrintQueueInfo(Q);
    while (1){

        printf("\nDequeue (y/n)? : ");
        scanf(" %c", &fix);

        if (fix == 'y' || fix == 'Y'){
            DelQueue(&Q, &pilihan);
        } else {
            break;
        };

        printf("Element yg dihapus : %d\n",
pilihan);

        PrintQueueInfo(Q);

        if (IsQueueEmpty(Q)){
            printf("Tekan apa saja..");
            getchar();
            getchar();
            break;
        };

    };
};

```

```
        break;

        case 3:
            printf("\n");
            PrintQueueInfo(Q);
            printf("Tekan apa saja...");
            getchar();
            getchar();
            break;
        break;

        default:

            break;

        case 0:

            break;
    }

    } while (pilihan != 0);
};
```

II.1.B. Screenshot



```
C:\Windows\system32\cmd.e  X  +  v  -  □  X
== Queue Circular Array ==
1. AddQueue
2. DelQueue
3. CetakQueue
Pilihan : 1

=== AddQueue ===
Masukan Element : 20
Lagi (y/n) : y
Masukan Element : 1
Lagi (y/n) : y
Masukan Element : 9
Lagi (y/n) : |
```

Gambar 2. 1 AddQueue Circular Array



```
C:\Windows\system32\cmd.e  X  +  v  -  □  X
== Queue Circular Array ==
1. AddQueue
2. DelQueue
3. CetakQueue
Pilihan : 3

Isi Queue : 20 1 9
Tekan apa saja...|
```

Gambar 2. 2 CetakQueue Circular Array



```
C:\Windows\system32\cmd.e  X  +  v  -  □  X
== Queue Circular Array ==
1. AddQueue
2. DelQueue
3. CetakQueue
Pilihan : 2

=== DelQueue ===
Isi Queue : 20 1 9

Deque (y/n)? : y
Element yg dihapus : 20
Isi Queue : 1 9

Deque (y/n)? : y
Element yg dihapus : 1
Isi Queue : 9

Deque (y/n)? : |
```

Gambar 2. 3 Dequeue Circular Array

II.1.C. Analisis

Queue Circular Array mengatasi masalah penuh semu pada antrean non-circular dengan memanfaatkan konsep array melingkar melalui operasi indeks sirkular. Saat fungsi AddQueue dijalankan, elemen baru dimasukkan dengan memajukan pointer TAIL satu langkah, di mana posisinya akan otomatis berputar kembali ke indeks awal (indeks 0) jika sudah mencapai batas maksimum array. Begitu pula pada fungsi DelQueue, data di posisi HEAD akan diambil dan dikosongkan, lalu pointer HEAD bergerak maju satu indeks. Jika pointer HEAD mencapai batas akhir array, posisinya akan ikut berputar kembali ke indeks 0 selama antrean tidak kosong. Melalui perputaran indeks ini, ruang kosong di bagian

depan array yang telah ditinggalkan dapat langsung digunakan kembali untuk menampung data baru secara optimal tanpa memerlukan pergeseran data.

II.2 Queue Circular LinkedList

II.2.A. Source Code

```
/* Program : boolean.h
Author : 2550081098, Faisal Fajari
Kelas : E
Deskripsi: Header file dari prototype Boolean
Tanggal : 06/04/2026
*/

#ifndef boolean_H
#define boolean_H
#define true 1
#define false 0
#define boolean unsigned char
#endif
```

```

/* Program : queuecircular2.h
Author : (2550081098, Faisal Fajari)
Kelas : E
Deskripsi: Header file dari prototype queue
Tanggal : 20/5/2026
*/

#ifndef _QUEUECIRCULAR_H
#define _ QUEUECIRCULAR_H
#include "boolean.h"
#include <stdio.h>
#define Nil NULL
#define MaxEl 2

#define Head(Q) (Q).HEAD
#define Tail(Q) (Q).TAIL
#define InfoHead(Q) (Q).T[(Q).HEAD]
#define InfoTail(Q) (Q).T[(Q).TAIL]

typedef int infotype;
typedef struct tElmStack *address;

typedef struct tElmStack{
    infotype Info;
    address next;
} ElmStack;

typedef struct {
    address HEAD;
    address TAIL;
    int counter;
} Queue;

void CreateQueue(Queue *Q);
int NBElt(Queue Q);
boolean IsQueueEmpty (Queue Q);
boolean IsQueueFull(Queue Q);
int NBElt(Queue Q);
void AddQueue (Queue *Q, infotype X);
void DelQueue (Queue *Q, infotype *X);

```



```
void PrintQueueInfo (Queue Q);  
boolean isInfoKetemu(Queue Q, infotype x);  
address CariElemenQueue(Queue Q, int X);  
  
#endif
```

```

/* Program : queuecircular2.c
Author : (2550081098, Faisal Fajari)
Kelas : E
Deskripsi: Header file dari prototype queue
Tanggal : 20/5/2026
*/
#include "boolean.h"
#include <stdio.h>
#include "queuecircular2.h"
#include <stdlib.h>

void CreateQueue(Queue *Q){
    Q->HEAD = Nil;
    Q->TAIL = Nil;
    Q->counter = 0;
};

address Alokasi(infotype x){
    address New = malloc(sizeof(ElmStack));
    New->Info = x;
    New->next = Nil;
    return New;
};

int NBElt(Queue Q){
    return (Q.counter);
};

boolean IsQueueEmpty (Queue Q){
    return (Q.counter == 0);
};

boolean IsQueueFull(Queue Q){
    return (Q.counter == MaxEl);
};

void AddQueue (Queue *Q, infotype X){

    address New = Alokasi(X);
    if (IsQueueEmpty(*Q)){
        Q->HEAD = New;
        Q->TAIL = New;
    }
}

```

```

        Q->TAIL->next = Q->HEAD;
        Q->counter = Q->counter + 1;
        return;
};

if (!IsQueueFull(*(Q))){
    Q->TAIL->next = New;
    Q->TAIL = New;
    Q->TAIL->next = Q->HEAD;
    Q->counter = Q->counter + 1;
    return;
};

printf("\nPenuh Le..");

};

void DelQueue (Queue *Q, infotype *X){
    if (IsQueueEmpty(*(Q))){
        printf("\nKosong");
        return;
    };

    address Temp = Q->HEAD;
    *X = Temp->Info;

    if (Q->HEAD == Q->TAIL){
        Q->HEAD = Nil;
        Q->TAIL = Nil;
        free(Temp);
        Q->counter = 0;
        return;
    };

    Q->HEAD = Q->HEAD->next;
    Q->TAIL->next = Q->HEAD;
    Q->counter = Q->counter - 1;

    free(Temp);

```

```

};

void PrintQueueInfo (Queue Q){
    if(IsQueueEmpty(Q)){
        printf("\nKosong\n");
        return;
    };

    address Temp = Q.HEAD;
    printf("Isi Queue : ");

    while(Temp->next != Q.HEAD){
        printf("%d ", Temp->Info);
        Temp = Temp->next;
    };
    printf("%d -> Repeat\n", Temp->Info);

};

boolean isInfoKetemu(Queue Q, infotype x){
    if(IsQueueEmpty(Q)){
        printf("\nKosong");
        return false;
    };

    address Temp = Q.HEAD;
    while(Temp->next != Q.HEAD){
        if (Temp->Info == x){
            return true;
        };
        Temp = Temp->next;
    };

    if (Temp->Info == x){
        return true;
    };

    return false;
};

```

```

};

address CariElemenQueue(Queue Q, int X){
    if(IsQueueEmpty(Q)){
        printf("\nKosong");
        return Nil;
    };

    address Temp = Q.HEAD;
    while(Temp->next != Q.HEAD){
        if(Temp->Info == X){
            return Temp;
        };
        Temp = Temp->next;
    };

    if(Temp->Info == X){
        return Temp;
    };

    return Nil;
};

```

```

/* Program : main.h
Author : 2550080198_Faisal Fajari
Kelas : E
Deskripsi: Main Driver file dari prototype queue
Tanggal : 18/5/26
*/

#include <stdio.h>
#include "queuecircular2.h"
#include <stdlib.h>

int main(){
    Queue Q;
    CreateQueue(&Q);
    int pilihan = 0;
    char fix;

    do {
        system("clear");
        system("cls");
        printf("1. AddQueue\n");
        printf("2. DelQueue\n");
        printf("3. CetakQueue\n");
        printf("Pilihan : ");
        scanf("%d",&pilihan);

        switch (pilihan) {
            case 1:
                printf("\n=== AddQueue ===\n");
                while (1){
                    printf("Masukan Element : ");
                    scanf("%d", &pilihan);

                    if (IsQueueFull(Q)){
                        printf("Queue Penuh...");
                        getchar();
                        getchar();
                        break;
                    }
                };
            case 2:
                printf("\n=== DelQueue ===\n");
                while (1){
                    printf("Masukan Element : ");
                    scanf("%d", &pilihan);

                    if (IsQueueEmpty(Q)){
                        printf("Queue Kosong...");
                        getchar();
                        getchar();
                        break;
                    }
                };
            case 3:
                printf("\n=== CetakQueue ===\n");
                while (1){
                    printf("Masukan Element : ");
                    scanf("%d", &pilihan);

                    if (IsQueueEmpty(Q)){
                        printf("Queue Kosong...");
                        getchar();
                        getchar();
                        break;
                    }
                };
            default:
                printf("Pilihan Salah\n");
                break;
        }
    } while (fix != 'q');
}

```

```

        AddQueue(&Q, pilihan);

        printf("Lagi (y/n) : ");
        scanf(" %c", &fix);

        if (fix == 'n' || fix == 'N'){
            break;
        };
    };
    break;

case 2:
    printf("\n=== DelQueue ===\n");
    PrintQueueInfo(Q);
    while (1){

        printf("\nDequeue (y/n)? : ");
        scanf(" %c", &fix);

        if (fix == 'y' || fix == 'Y'){
            DelQueue(&Q, &pilihan);
        } else {
            break;
        };

        printf("Element yg dihapus : %d\n",
pilihan);

        PrintQueueInfo(Q);

        if (IsQueueEmpty(Q)){
            printf("Tekan apa saja..");
            getchar();
            getchar();
            break;
        };

    };
};

```

```
        break;

        case 3:
            printf("\n");
            PrintQueueInfo(Q);
            printf("Tekan apa saja...");
            getchar();
            getchar();
            break;
        break;

        default:

            break;

        case 0:

            break;
    }

    } while (pilihan != 0);
};
```


II.2.B. Screenshot

```
C:\Windows\system32\cmd.e  X  +  v
== Queue Circular Linked List ==
1. AddQueue
2. DelQueue
3. CetakQueue
Pilihan : 1

=== AddQueue ===
Masukan Element : 20
Lagi (y/n) : y
Masukan Element : 90
Lagi (y/n) : y
Masukan Element : 80
Lagi (y/n) : |
```

Gambar 2. 4 AddQueue Circular Linked_List

```
C:\Windows\system32\cmd.e  X  +  v
== Queue Circular Linked List ==
1. AddQueue
2. DelQueue
3. CetakQueue
Pilihan : 3

Isi Queue : 20 90 80 -> Repeat
Tekan apa saja...|
```

Gambar 2. 5 CetakQueue Circular Linked_List

```
C:\Windows\system32\cmd.e  X  +  v
== Queue Circular Linked List ==
1. AddQueue
2. DelQueue
3. CetakQueue
Pilihan : 2

=== DelQueue ===
Isi Queue : 20 90 80 -> Repeat

Dequeue (y/n)? : y
Element yg dihapus : 20
Isi Queue : 90 80 -> Repeat

Dequeue (y/n)? : y
Element yg dihapus : 90
Isi Queue : 80 -> Repeat

Dequeue (y/n)? : |
```

Gambar 2. 6 Dequeue Circular Linked_List

II.2.C. Analisis

Queue Circular LinkedList mengimplementasikan antrean melingkar secara dinamis menggunakan pointer, di mana node terakhir (TAIL) dipaksa untuk selalu menunjuk kembali ke node pertama (HEAD). Pada fungsi AddQueue, elemen baru dimasukkan dengan mengalokasikan memori node baru, menyambungkannya di belakang node TAIL lama, memindahkan label TAIL ke node baru tersebut, lalu mengarahkan pointer next-nya kembali mengunci posisi HEAD. Untuk fungsi DelQueue, penghapusan dilakukan dengan menandai node HEAD lama ke pointer bantuan, memajukan pointer HEAD utama ke node berikutnya, lalu memperbarui pointer next dari TAIL agar terhubung ke HEAD yang baru sebelum akhirnya menghapus memori node lama dengan perintah free.

BAB III. KESIMPULAN

Berdasarkan praktikum yang dilakukan, struktur data Queue secara menerapkan prinsip First In First Out (FIFO), di mana data yang masuk pertama melalui AddQueue akan keluar pertama melalui DelQueue. Karakteristik penambahan dan penghapusan data ini sangat bergantung pada media penyimpanan memori yang digunakan. Array statis non-circular memaksa kita memilih antara mengorbankan performa kecepatan (Alternatif 1) atau mengorbankan ruang memori karena masalah penuh semu (Alternatif 2). Solusi terbaik untuk keterbatasan array statis tersebut adalah menerapkan logika Queue Circular Array yang memutar kembali indeks ke awal , atau menggunakan Queue Circular LinkedList yang jauh lebih dinamis karena ukurannya bisa bertambah dan berkurang secara otomatis di memori komputer tanpa batasan kapasitas yang kaku.